МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Сопроводительная записка

по производственной практике

к программному модулю «Модуль импорта товаров в электронный магазин из CSV-файлов»

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей
(индекс по РУП и наименование профессионального модуля)

Специальность 09.02.07 Информационные системы и программирование

Обучающийся _____ Ощепков Михаил Алексеевич
(подпись) (фамилия, имя, отчество)

Группа __ИПо-42_____

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«13» ноября 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

1. ВВЕДЕНИЕ.....3
2. ЦЕЛИ И ЗАДАЧИ.....3
3. ОПИСАНИЕ МОДЕЛИ ДАННЫХ.....4
3.1. Product (товар).....4
3.2. ImportResult (результат импорта).....4
3.3. ImportError (ошибка импорта).....4
4. ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ.....5
4.1. Технологии.....5
4.2. Архитектура.....5
5. ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ.....6
5.1. Обязательные функции.....6
5.2. Дополнительные функции.....6
5.3. Правила проверки полей.....7
6. ФОРМАТ CSV-ФАЙЛА.....7
7. ЭТАПЫ РАЗРАБОТКИ.....8
7.1. Создание проекта.....8
7.2. Модель данных и репозиторий.....8
7.3. Чтение CSV и валидация.....9
7.4. Интеграция в MVC-приложение.....10
7.5. Контроль версий.....13
8. ТЕСТИРОВАНИЕ.....14
9. ИНСТРУКЦИЯ ПО ЗАПУСКУ.....15
10. ЗАКЛЮЧЕНИЕ.....16

1. ВВЕДЕНИЕ

Настоящая сопроводительная записка описывает программный модуль «Модуль импорта товаров в электронный магазин из CSV-файлов», разработанный в ходе производственной практики ПП.02.01 в АО «ЭР-Телеком Холдинг» (вариант индивидуализации 1 «Электроника»).

Модуль предназначен для наполнения каталога интернет-магазина абонентского оборудования: он читает CSV-файл с данными о товарах, проверяет каждую строку, добавляет корректные товары в коллекцию в памяти приложения и выводит результат импорта со списком ошибок по номерам строк. Модуль реализован как веб-приложение ASP.NET Core MVC на языке C#; база данных не используется, данные хранятся до перезапуска приложения.

Записка содержит описание модели данных, требования к модулю, формат входного файла, этапы разработки с основными фрагментами кода, сведения о тестировании и инструкцию по запуску.

2. ЦЕЛИ И ЗАДАЧИ

Цель — разработать модуль импорта товаров из CSV-файла с валидацией данных и хранением товаров в памяти приложения. Задачи:

- подготовить описание формата CSV-файла и правил проверки полей;
- спроектировать модель товара, результата импорта и хранилище в памяти;
- реализовать чтение CSV-файла с поддержкой кавычек, разных разделителей и кодировок;
- реализовать валидацию данных и контроль дубликатов артикула;
- интегрировать модуль в MVC-приложение: страницы импорта, списка товаров и карточки товара;
- подготовить тестовые CSV-файлы и модульные тесты xUnit;
- разместить исходный код в системе контроля версий Git.

3. ОПИСАНИЕ МОДЕЛИ ДАННЫХ

3.1. Product (товар)

- Sku — артикул товара, уникальный;
- Name, Description — название и описание;
- Price, Currency — цена и валюта (по умолчанию RUB);
- CategoryName — категория (по умолчанию «Без категории»);
- StockQuantity — остаток на складе;
- ImageUrl — ссылка на изображение;
- IsActive — признак активности товара;
- Brand, WarrantyMonths, ModelNumber — бренд, гарантия в месяцах и номер модели (поля варианта «Электроника»).

3.2. ImportResult (результат импорта)

- TotalRows — количество обработанных строк с данными;
- ImportedCount — количество добавленных товаров;
- Errors, ErrorCount — список ошибок и их количество;
- Delimiter, EncodingName — разделитель и кодировка, с которыми был прочитан файл.

3.3. ImportError (ошибка импорта)

- RowNumber — номер строки файла (нумерация с 1, строка 1 — заголовок);
- Message — описание ошибки.

Связи между классами показаны на рисунке 1.

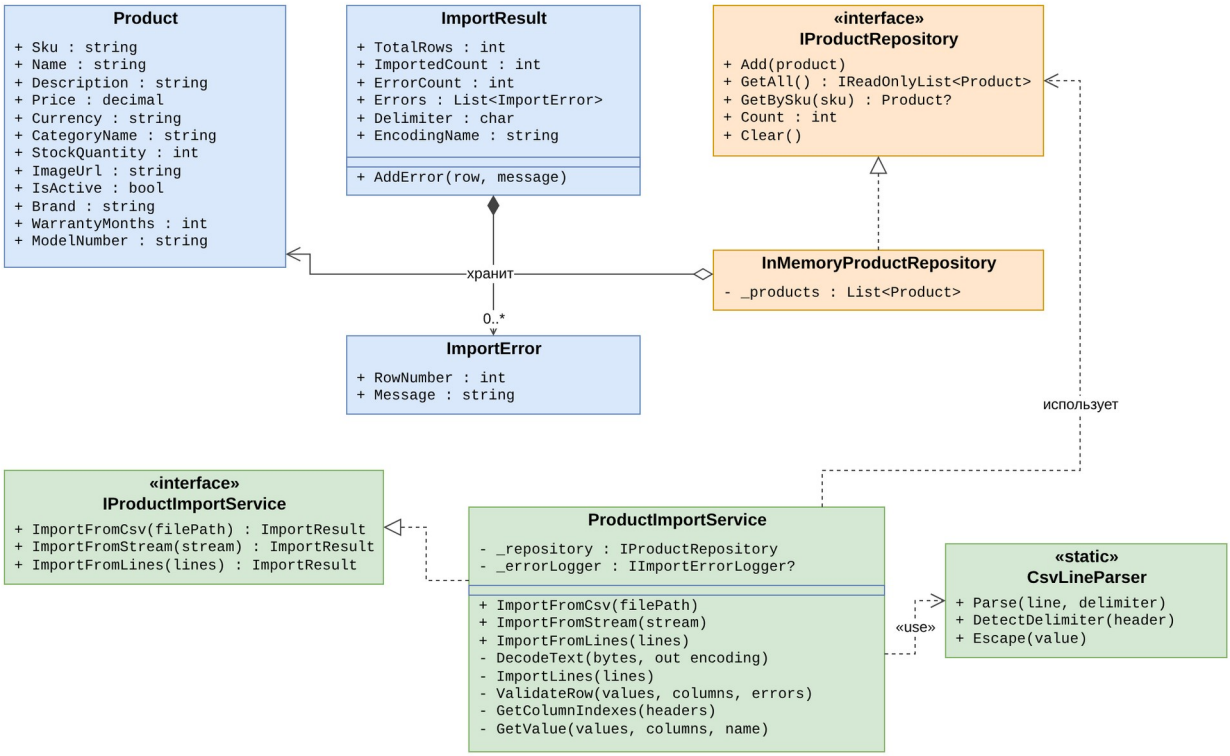


Рисунок 1 – Диаграмма классов модуля

4. ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ

4.1. Технологии

- Backend: ASP.NET Core MVC (.NET 8), C#;

- представления: Razor, Bootstrap 5;

- хранение данных: коллекция List<Product> в памяти приложения;

- тестирование: xUnit;

- среда разработки: Visual Studio 2022; контроль версий: Git.

4.2. Архитектура

Приложение построено по шаблону MVC с выделенным сервисным слоем (рисунок 2):

- контроллер ProductsController — принимает запросы и передает результат в представления, бизнес-логики не содержит;

- сервис ProductImportService — чтение файла, разбор строк, валидация, формирование ImportResult;

- CsvLineParser — разбор одной строки CSV на поля;

- репозиторий InMemoryProductRepository — хранение товаров (Singleton);

- представления Import, Index, Details — интерфейс пользователя.

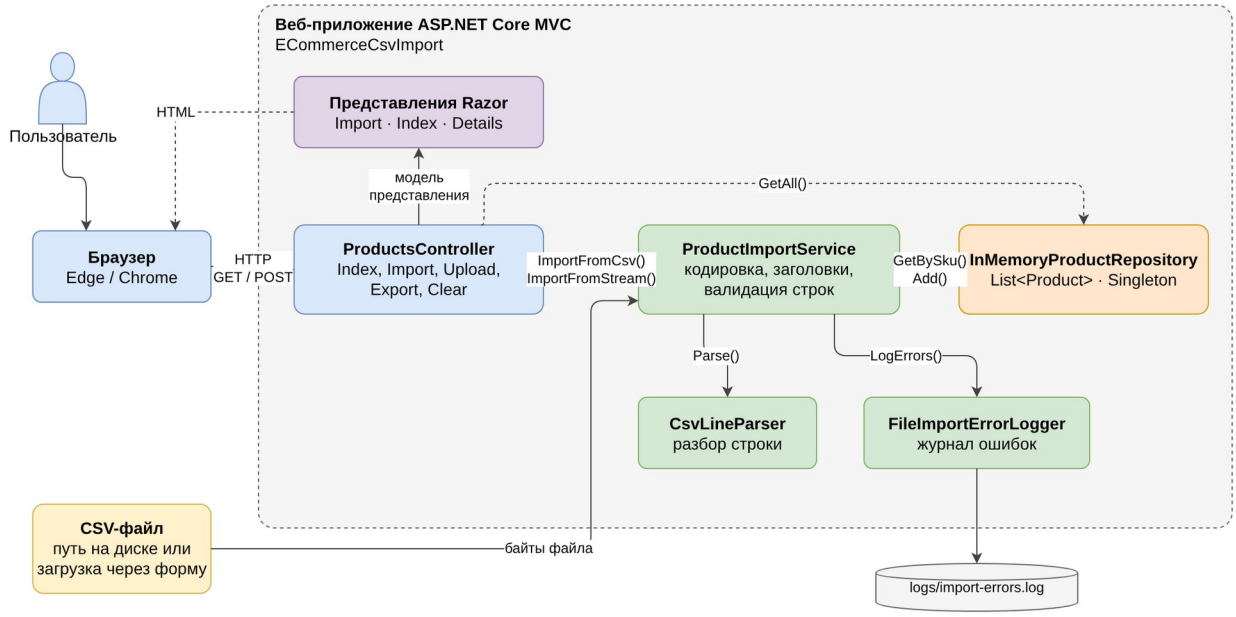


Рисунок 2 – Архитектура приложения

5. ФУНКЦИОНАЛЬНЫЕ ТРЕБОВАНИЯ

5.1. Обязательные функции

- автоматический импорт CSV-файла при запуске приложения; путь к файлу задается в appsettings.json (раздел Import);

- импорт файла по пути, указанному пользователем на странице «Импорт»;

- чтение CSV-файла построчно и преобразование строк в объекты Product;

- проверка заголовков, количества полей и значений каждого поля;

- пропуск строк с ошибками без остановки импорта остальных строк;

- пропуск дубликатов артикула (в файле и среди уже загруженных товаров);

- вывод результата: всего строк, импортировано, ошибок, таблица ошибок «Строка — Описание»;

- просмотр списка загруженных товаров; повторный импорт без падения приложения.

5.2. Дополнительные функции

- загрузка файла через браузер (только .csv, до 5 МБ);

- поиск по названию, фильтр по категории, сортировка по цене, остатку и названию;

- карточка товара, подсчет общего количества и общей стоимости товаров;

- экспорт списка товаров в CSV (файл можно импортировать обратно);

- запись ошибок импорта в журнал logs/import-errors.log;

- автоопределение разделителя («,», «;», табуляция) и кодировки (UTF-8, Windows-1251).

5.3. Правила проверки полей

Правила проверки и тексты сообщений об ошибках приведены в таблице 1.

Таблица 1 – Правила проверки полей CSV-файла

Поле	Правило	Сообщение при нарушении
sku	не пустое, уникальное	Поле sku не может быть пустым; Артикул товара уже существует
name	не пустое	Поле name не может быть пустым
price	число больше 0	Поле price должно быть числом больше 0
stock	целое, не меньше 0	Остаток на складе должен быть целым числом не меньше 0
is_active	true, false, 1 или 0	Поле is_active должно иметь значение true, false, 1 или 0
brand	не пустое	Поле brand обязательно для варианта «Электроника»
warranty_months	целое от 0 до 60	Гарантия должна быть целым числом от 0 до 60 месяцев
description, image_url, model_number	могут быть пустыми	—
currency / category_name	пусто → RUB / «Без категории»	—

Ошибки уровня файла: «Файл не найден», «Данные для импорта отсутствуют», «Неверные заголовки CSV», «Неверное количество полей в строке».

6. ФОРМАТ CSV-ФАЙЛА

Первая строка содержит заголовки колонок, каждая следующая строка — один товар. Порядок колонок может быть любым, так как поиск

выполняется по названию заголовка. Значение, содержащее разделитель, заключается в кавычки. Пример:

```
sku,name,description,price,currency,category_name,stock,image_url,is_active,b
rand,warranty_months,model_number
1001,Wi-Fi роутер Дом.ру Wave Plus,Роутер Wi-Fi 6 AX3000 для
квартиры,3990,RUB,Роутеры,25,/images/wave-plus.jpg,true,Дом.ру,12,Wave Plus
AX3000
```

Пример основного файла находится в проекте по пути wwwroot/csv/sample_products.csv (11 товаров: роутеры Дом.ру и TP-Link, ТВ-приставки Movix, камеры Omny, устройства умного дома, кабель). Файлы для проверки ошибок — в папке wwwroot/csv/test.

7. ЭТАПЫ РАЗРАБОТКИ

7.1. Создание проекта

В Visual Studio создано решение ECommerceCsvImport с двумя проектами: веб-приложение ASP.NET Core MVC (src/ECommerceCsvImport) и проект тестов xUnit (tests/ECommerceCsvImport.Tests). Подготовлены README.md и тестовые CSV-файлы.

7.2. Модель данных и репозиторий

Создан класс товара (листинг 1).

Листинг 1 – Класс Product

```
namespace ECommerceCsvImport.Models
{
    // Товар электронного магазина (вариант 1 - Электроника)
    public class Product
    {
        public string Sku { get; set; } = string.Empty;
        public string Name { get; set; } = string.Empty;
        public string Description { get; set; } = string.Empty;
        public decimal Price { get; set; }
        public string Currency { get; set; } =
ImportConstants.DefaultCurrency;
        public string CategoryName { get; set; } =
ImportConstants.DefaultCategory;
        public int StockQuantity { get; set; }
        public string ImageUrl { get; set; } = string.Empty;
        public bool IsActive { get; set; }

        // Дополнительные поля для варианта "Электроника"
        public string Brand { get; set; } = string.Empty;
        public int WarrantyMonths { get; set; }
        public string ModelNumber { get; set; } = string.Empty;
    }
}
```

Интерфейс хранилища приведен в листинге 2. Реализация InMemoryProductRepository хранит товары в List<Product>, поиск по артикулу выполняется без учета регистра и пробелов по краям.

Листинг 2 – Интерфейс IProductRepository

```
using ECommerceCsvImport.Models;

namespace ECommerceCsvImport.Services
{
    // Интерфейс хранилища товаров
    public interface IProductRepository
    {
        void Add(Product product);
        IReadOnlyList<Product> GetAll();
        Product? GetBySku(string sku);
        int Count { get; }
        void Clear();
    }
}
```

7.3. Чтение CSV и валидация

Класс CsvLineParser разбирает строку посимвольно: учитывает поля в кавычках и экранированные кавычки (две кавычки подряд). Сервис импорта реализует интерфейс из листинга 3.

Листинг 3 – Интерфейс IProductImportService

```
using ECommerceCsvImport.Models;

namespace ECommerceCsvImport.Services
{
    // Интерфейс сервиса импорта товаров из CSV
    public interface IProductImportService
    {
        // Импорт из файла на диске
        ImportResult ImportFromCsv(string filePath);

        // Импорт из файла, загруженного через браузер
        ImportResult ImportFromStream(Stream stream);

        // Импорт из набора строк (удобно для unit-тестов)
        ImportResult ImportFromLines(IEnumerable<string> lines);
    }
}
```

Проверка полей строки выполняется в методе ValidateRow: все ошибки строки собираются в список, и если он пустой, товар добавляется в репозиторий. Начало метода показано в листинге 4.

Листинг 4 – Фрагмент метода ValidateRow

```
private Product ValidateRow(List<string> values, Dictionary<string, int>
columns, List<string> errors)
{
    Product product = new Product();

    product.Sku = GetValue(values, columns, "sku");
    if (product.Sku == "")
        errors.Add(ImportMessages.EmptySku);

    product.Name = GetValue(values, columns, "name");
    if (product.Name == "")
        errors.Add(ImportMessages.EmptyName);

    // Цена: допускаем и точку, и запятую как разделитель дробной части
    string priceText = GetValue(values, columns, "price").Replace(',', '.');
    decimal price;
    if (decimal.TryParse(priceText, NumberStyles.Number,
CultureInfo.InvariantCulture, out price) && price > 0)
        product.Price = price;
    else
        errors.Add(ImportMessages.InvalidPrice);
```

7.4. Интеграция в MVC-приложение

Сервисы регистрируются в Program.cs (листинг 5). Репозиторий зарегистрирован как Singleton, чтобы товары сохранялись между HTTP-запросами.

Листинг 5 – Регистрация сервисов в Program.cs

```
// Репозиторий и сервис регистрируем как Singleton: товары хранятся в памяти
// и должны сохраняться между запросами (при AddScoped они бы терялись).
builder.Services.AddSingleton<IProductRepository,
InMemoryProductRepository>();
builder.Services.AddSingleton<IImportErrorLogger>(
    new
FileImportErrorLogger(Path.Combine(builder.Environment.ContentRootPath,
"logs", "import-errors.log")));
builder.Services.AddSingleton<IProductImportService, ProductImportService>();
```

Сразу после сборки приложения в Program.cs выполняется импорт файла, путь к которому берется из appsettings.json (листинг 6). Поэтому после запуска каталог уже заполнен.

Листинг 6 – Импорт при запуске приложения

```
// Автоматический импорт товаров при запуске приложения.
// Путь к файлу (относительно папки wwwroot) задается в appsettings.json,
раздел "Import".
bool autoImport =
builder.Configuration.GetValue<bool>("Import:AutoImportOnStartup");
string? importPath = builder.Configuration["Import:FilePath"];
if (autoImport && !string.IsNullOrEmpty(importPath))
{
    string fullPath = Path.Combine(app.Environment.WebRootPath, importPath);
    IProductImportService importService =
app.Services.GetRequiredService<IProductImportService>();
    ImportResult startupResult = importService.ImportFromCsv(fullPath);
    app.Logger.LogInformation("Импорт при запуске ({Path}): добавлено
{Imported}, ошибок {Errors}",
        importPath, startupResult.ImportedCount, startupResult.ErrorCount);
}
```

Контроллер ProductsController содержит действия Index, Details, Import, ImportPost, Upload, Export и Clear. Непредвиденные исключения перехватываются, и сообщение выводится на странице.

Страница импорта с результатом показана на рисунке 3, список товаров — на рисунке 4.

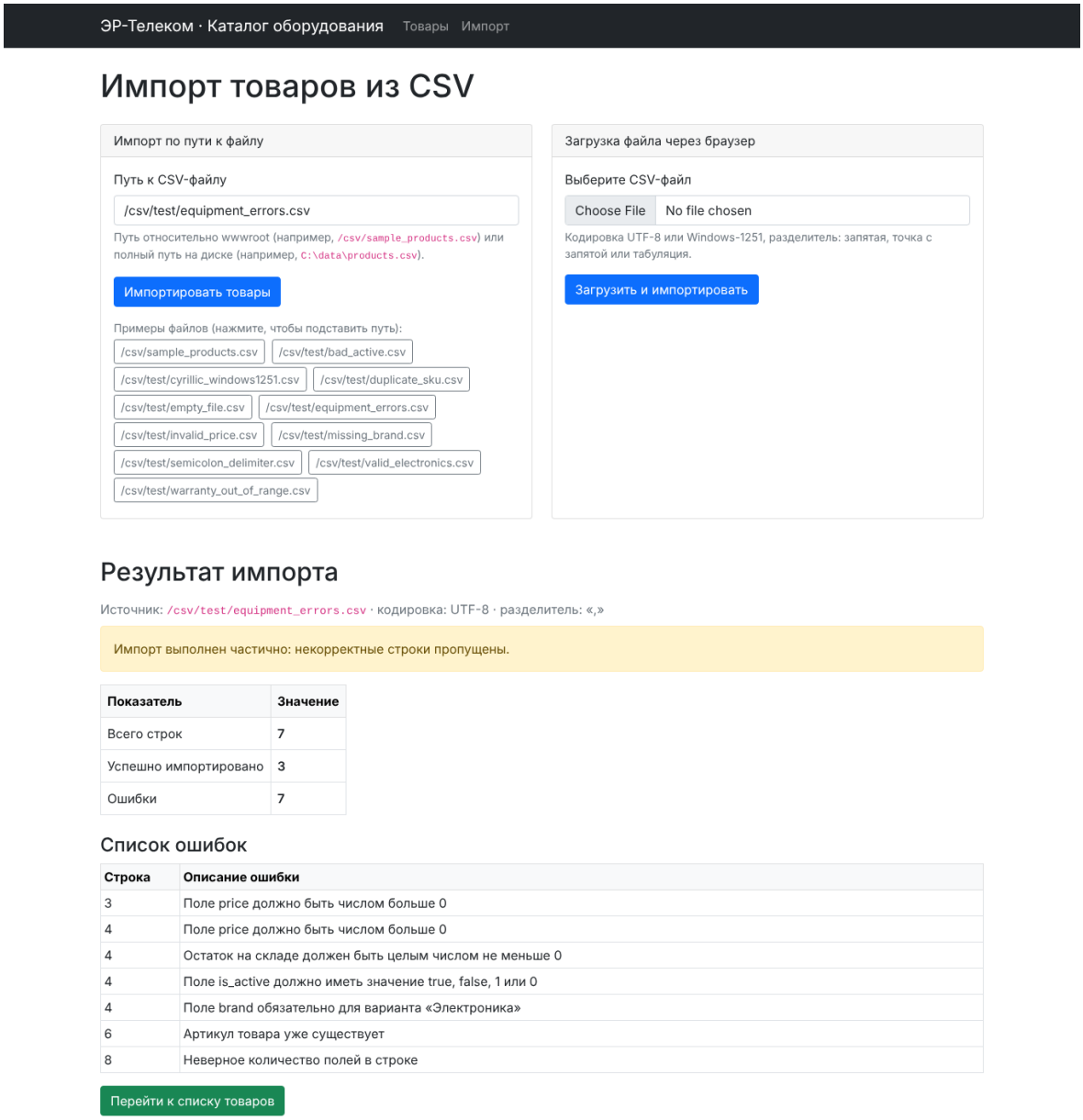


Рисунок 3 – Результат импорта файла с ошибками

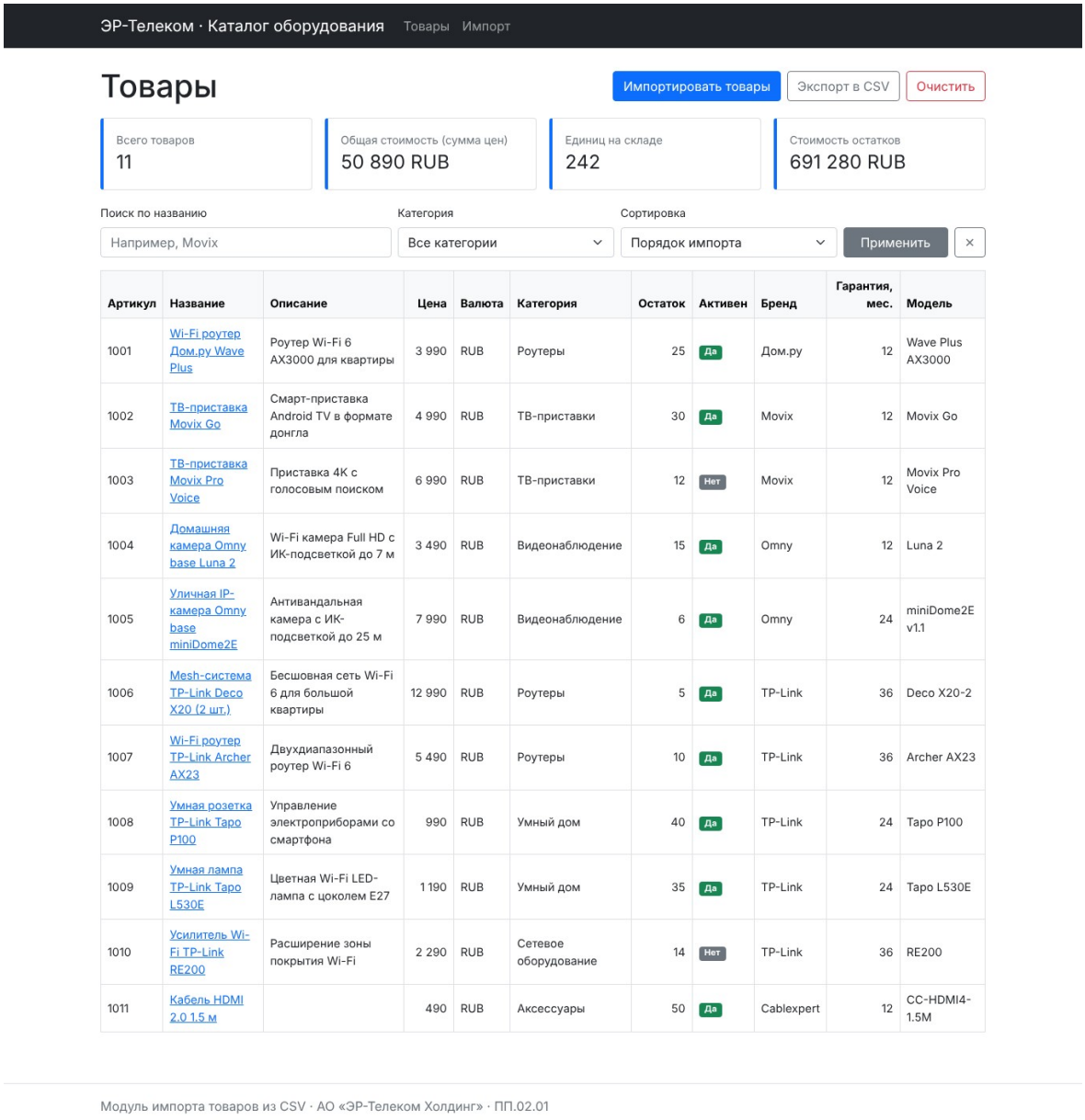


Рисунок 4 – Список товаров после импорта

7.5. Контроль версий

Каждый этап зафиксирован отдельным коммитом (листинг 7).

Листинг 7 – История коммитов

```
55e880a Add project scaffold and CSV fixtures
c2b45f3 Add product model and in-memory repository
d937b26 Add CSV line parser with quotes support
e997006 Add product import service with validation for electronics
a9cc995 Add products controller and import views
0d3f077 Add unit tests for repository, parser and import service
64c381f Import sample products on application startup
```

Репозиторий: https://github.com/ЛОГИН/ECommerceCsvImport

8. ТЕСТИРОВАНИЕ

Тестовые CSV-файлы: valid_electronics, invalid_price, duplicate_sku, missing_brand, warranty_out_of_range, bad_active, empty_file, equipment_errors, cyrillic_windows1251, semicolon_delimiter. Каждый файл проверяет одно правило или ситуацию.

Модульные тесты написаны на xUnit по плану тестирования из задания (таблица 2).

Таблица 2 – Модульные тесты

Класс тестов	**Кол-во**	**Что проверяется**
RepositoryTests	4	добавление, поиск по SKU, поиск без учета регистра, пустой репозиторий
CsvLineParserTests	4	простая строка, пустые поля, запятая в кавычках, экранированные кавычки
ProductImportServiceTests	23	успешный импорт, ошибки файла и заголовков, все правила валидации, дубликаты, значения по умолчанию, кириллица
Итого	31	

Все тесты выполняются успешно (рисунок 5).

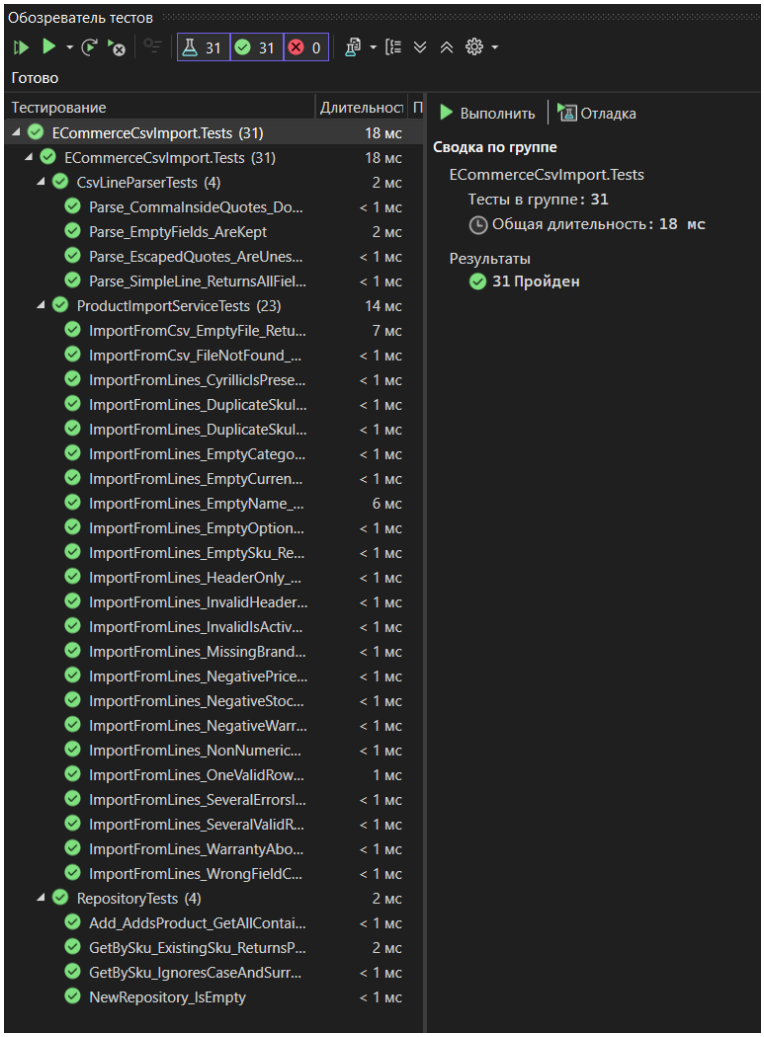


Рисунок 5 – Результат запуска тестов в Visual Studio

9. ИНСТРУКЦИЯ ПО ЗАПУСКУ

1. Установить .NET 8 SDK и Visual Studio 2022 (рабочая нагрузка «ASP.NET и разработка веб-приложений»).

2. Открыть файл решения ECommerceCsvImport.sln и выбрать проект ECommerceCsvImport как запускаемый.

3. Запустить приложение: Ctrl+F5 (без отладки) или F5 (с отладкой). Откроется страница «Товары» — при запуске файл sample_products.csv уже импортирован автоматически, в каталоге 11 товаров.

4. Чтобы проверить импорт вручную, нажать «Очистить», перейти на страницу «Импорт», оставить путь /csv/sample_products.csv и нажать «Импортировать товары» — будет загружено 11 товаров без ошибок.

5. Для проверки ошибок указать путь /csv/test/equipment_errors.csv или выбрать любой файл из папки wwwroot/csv/test через форму загрузки.

6. Тесты запускаются через меню «Тест → Обозреватель тестов → Запустить все тесты» или командой dotnet test.

Примечание. В некоторых версиях Visual Studio при запуске с отладкой (F5) в браузере Google Chrome отладка может завершаться в момент выбора файла для загрузки. Это известная ошибка связки отладчика Visual Studio с браузерами на движке Chromium, к коду модуля она не относится. В этом случае следует запускать приложение без отладки (Ctrl+F5) или выбрать в Visual Studio браузер Microsoft Edge.

10. ЗАКЛЮЧЕНИЕ

Разработан модуль импорта товаров в электронный магазин из CSV-файлов для варианта «Электроника». Модуль выполняет все обязательные требования задания и все 10 дополнительных функций: загрузку файла через браузер, поиск, фильтрацию, сортировку, карточку товара, подсчет итогов, экспорт в CSV, модульные тесты, журнал ошибок и поддержку нескольких разделителей. Работа модуля проверена ручными сценариями и 31 модульным тестом. Каталог заполнен оборудованием, которое АО «ЭР-Телеком Холдинг» предлагает абонентам под брендом «Дом.ру».